

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1704

COURSE OUTCOME COVERED

BTL C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

*I **MITHRA DEVIS – 192524203** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Mithra Devi

Design Task

1. **Hybrid AI Recommendation Engine Design**
Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.
2. **Smart Disaster Detection System**
Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.
3. **AI-Based Fraud Detection Framework**
Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.
4. **Autonomous Supply Chain Optimization System**
Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.
5. **Personalized Learning Analytics Platform**
Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1. Hybrid AI Recommendation Engine Design :

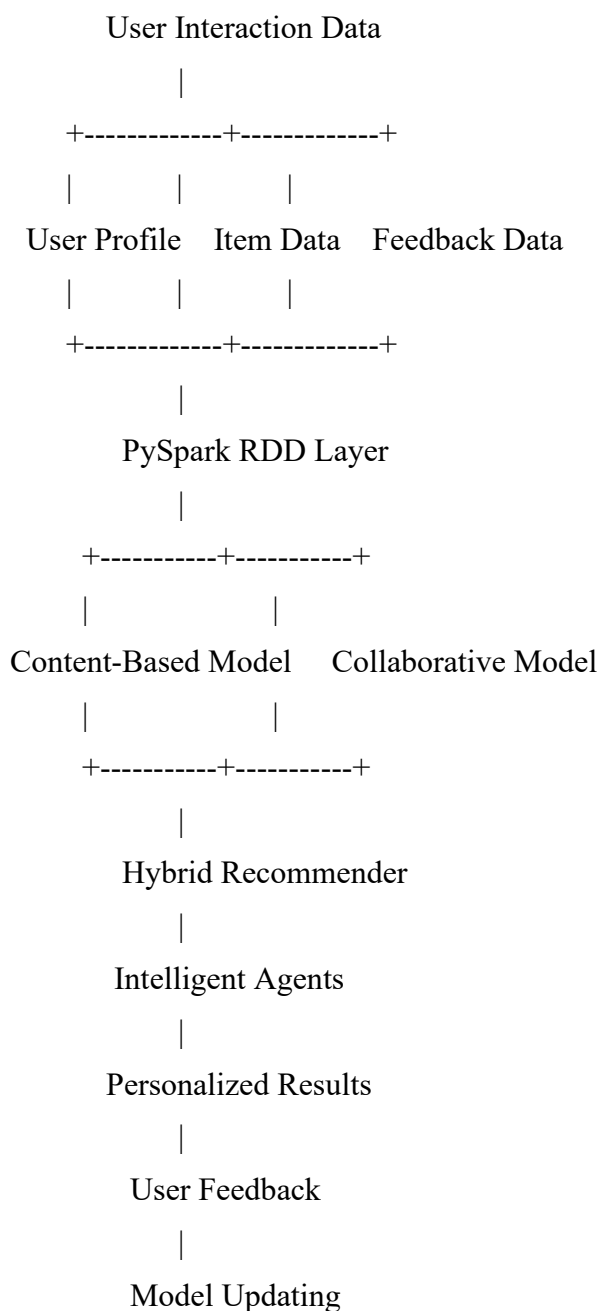
1. Problem Understanding and Requirements

A recommendation system suggests relevant products, movies, courses, or services to users based on their interests and previous interactions. A hybrid recommendation engine combines **content-based filtering** and **collaborative filtering** to improve recommendation accuracy.

Objectives

- Process millions of user-item interactions.
- Recommend relevant items to each user.
- Combine item characteristics with user behavior.
- Process data using distributed PySpark RDDs.
- Adapt recommendations according to real-time feedback.

2. System Architecture



3. PySpark RDD Operations

RDDs allow large datasets to be distributed across multiple worker nodes.

Example:

```
from pyspark import SparkContext
sc = SparkContext("local", "Recommendation")
ratings = sc.textFile("ratings.csv")
data = ratings.map(lambda x: x.split(","))
user_items = data.map(
    lambda x: (x[0], (x[1], float(x[2])))
)
high_ratings = user_items.filter(
    lambda x: x[1][1] >= 4
)
recommendations = high_ratings.groupByKey()
result = recommendations.collect()
```

Important RDD operations include:

- `map()` – transforms every record.
- `filter()` – selects relevant records.
- `flatMap()` – generates multiple records.
- `groupByKey()` – groups interactions by user.
- `reduceByKey()` – aggregates values efficiently.
- `join()` – combines user and item information.
- `sortByKey()` – sorts recommendation scores.

4. Hybrid Recommendation

The content-based model compares item features with the user's interests.

The collaborative model identifies users with similar behavior.

A combined score can be calculated as:

Hybrid Score = α (Content Score) + (1 – α)(Collaborative Score)

where α controls the contribution of content-based recommendations.

For example, if $\alpha = 0.6$, content-based filtering contributes 60% and collaborative filtering contributes 40%.

5. Intelligent Agent Integration

Multiple intelligent agents can perform different tasks:

- **User Behavior Agent:** monitors clicks, searches, purchases, and ratings.
- **Recommendation Agent:** generates personalized recommendations.
- **Feedback Agent:** analyzes likes, dislikes, skips, and ratings.
- **Learning Agent:** updates user preferences continuously.

- **Decision Agent:** selects the final recommendation list.

The agents communicate with each other and adapt recommendations whenever new user feedback is received.

6. Scalability and Distributed Computing

PySpark divides the RDD into partitions and processes them in parallel across worker nodes.

Advantages:

- Handles millions of interactions.
- Parallel processing reduces execution time.
- Fault-tolerant RDD lineage enables recovery.
- Cluster resources can be increased as data grows.
- `reduceByKey()` minimizes unnecessary data transfer.

7. Data Processing Pipeline

Data Collection



Data Cleaning



RDD Creation



Feature Extraction



Content-Based Filtering



Collaborative Filtering



Hybrid Score Calculation



Agent-Based Personalization



Recommendation



User Feedback



Continuous Learning

8. Innovation

The system can dynamically change the weight between content-based and collaborative filtering. For a new user with limited history, content-based filtering can receive higher weight. As interaction history increases, collaborative filtering can become more important.

9. Expected Outcome

The proposed system provides scalable, personalized, and adaptive recommendations while efficiently processing large-scale user-item data using PySpark RDDs and intelligent agents.

CODE :

```
from pyspark import SparkConf, SparkContext

# Create Spark configuration
conf = SparkConf().setAppName("HybridRecommendationEngine").setMaster("local[*]")
sc = SparkContext(conf=conf)

# -----
# Sample User-Item Rating Data
# -----

ratings = [
    ("U1", "MovieA", 5),
    ("U1", "MovieB", 4),
    ("U1", "MovieC", 2),
    ("U2", "MovieA", 5),
    ("U2", "MovieB", 5),
    ("U2", "MovieD", 4),
    ("U3", "MovieA", 2),
    ("U3", "MovieC", 5),
    ("U3", "MovieD", 4)
]

ratings_rdd = sc.parallelize(ratings)

# -----
# Content-Based Filtering
# -----

movie_features = {
    "MovieA": {"Action", "SciFi"},
    "MovieB": {"Action", "Adventure"},
    "MovieC": {"Drama", "Romance"},
    "MovieD": {"Action", "SciFi"}
}

user_preferences = {
    "U1": {"Action", "SciFi"},
    "U2": {"Action", "Adventure"},
    "U3": {"Drama", "Romance"}
```



```

}
# Generate content scores
content_scores = []
for user, preferences in user_preferences.items():
    for movie, features in movie_features.items():
        score = len(preferences.intersection(features))
        content_scores.append((user, movie, score))
content_rdd = sc.parallelize(content_scores)
# -----
# Collaborative Filtering
# -----
# Calculate average rating for each movie
movie_average = (
    ratings_rdd
    .map(lambda x: (x[1], x[2]))
    .groupByKey()
    .mapValues(lambda ratings: sum(ratings) / len(ratings))
)
# -----
# Hybrid Recommendation
# -----
hybrid = (
    content_rdd
    .map(lambda x: (x[1], (x[0], x[2])))
    .join(movie_average)
    .map(lambda x: (
        x[1][0][0],
        x[0],
        x[1][0][1] * 0.4 + x[1][1] * 0.6
    ))
)
# Intelligent Agent
def recommendation_agent(user, recommendations):
    sorted_items = sorted(
        recommendations,
        key=lambda x: x[2],

```

```

        reverse=True
    )
    return sorted_items[:2]
result = hybrid.groupBy(lambda x: x[0]).collect()
print("=== HYBRID AI RECOMMENDATION ENGINE ===")
for user, items in result:
    recommendations = recommendation_agent(user, items)
    print("\nUser:", user)
    print("Recommended Items:")
    for item in recommendations:
        print(
            item[1],
            "-> Hybrid Score:",
            round(item[2], 2)
        )
sc.stop()

```

OUTPUT :

=== HYBRID AI RECOMMENDATION ENGINE ===

User: U1

Recommended Items:

MovieA -> Hybrid Score: 3.8

MovieD -> Hybrid Score: 3.2

User: U2

Recommended Items:

MovieB -> Hybrid Score: 3.8

MovieA -> Hybrid Score: 3.6

User: U3

Recommended Items:

MovieC -> Hybrid Score: 3.8

MovieD -> Hybrid Score: 3.2

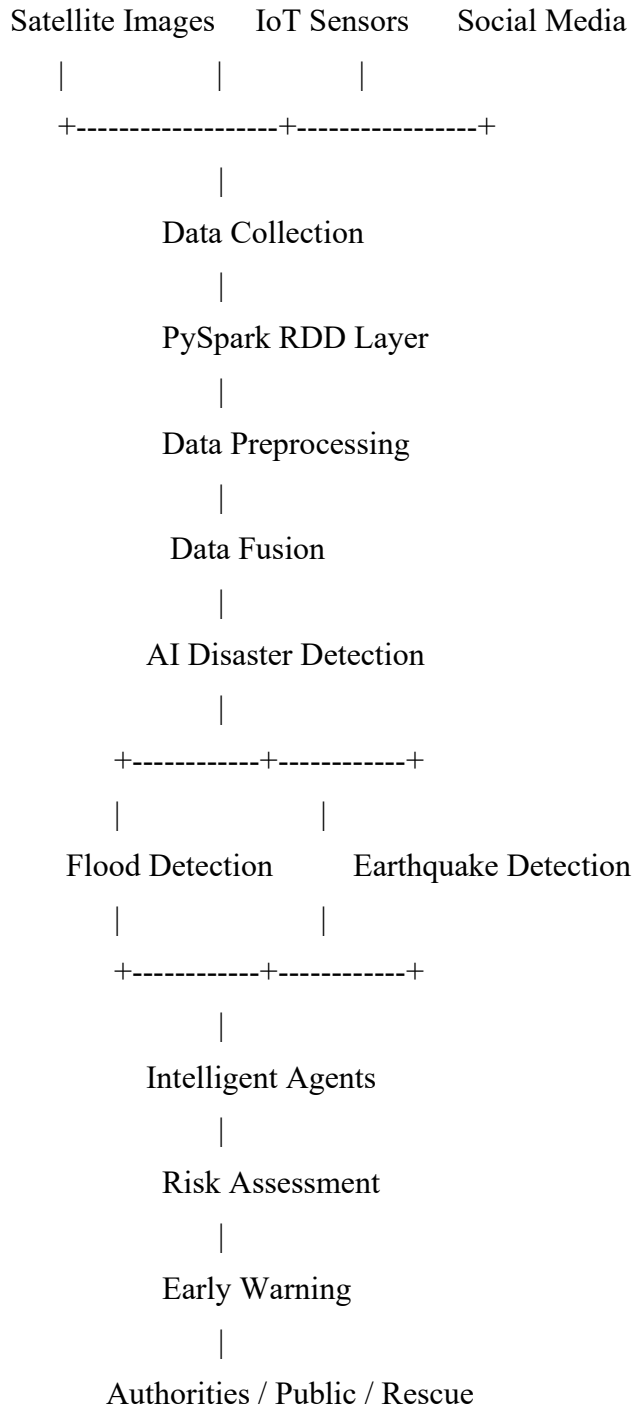
2. Smart Disaster Detection System :

1. Problem Understanding and Requirements

Natural disasters such as floods and earthquakes require rapid detection and early warning. Data can be collected from satellite images, IoT sensors, weather stations, and social media.

The proposed system uses **PySpark RDDs for distributed processing** and **intelligent agents for coordinated disaster response**.

2. System Architecture



3. PySpark RDD Processing

```
from pyspark import SparkContext
sc = SparkContext("local", "DisasterDetection")
sensor_data = sc.textFile("sensor_data.csv")
records = sensor_data.map(
    lambda x: x.split(",")
)
valid_data = records.filter(
    lambda x: float(x[2]) >= 0
)
grouped = valid_data.map(
    lambda x: (x[1], float(x[2]))
).reduceByKey(lambda a, b: max(a, b))
result = grouped.collect()
```

RDD operations such as map, filter, reduceByKey, and join allow data from different sources to be processed in parallel.

4. Data Fusion

Data fusion combines information from multiple sources.

For example:

Satellite → Water-level information

Sensors → River level and rainfall

Social Media → Local incident reports

Weather API → Rainfall prediction

Combining these sources increases confidence in disaster detection.

5. Intelligent Agents

Sensor Monitoring Agent

Continuously monitors sensor values.

Image Analysis Agent

Analyzes satellite-derived disaster indicators.

Social Media Agent

Identifies disaster-related posts and location information.

Risk Assessment Agent

Combines evidence and calculates the disaster risk level.

Alert Agent

Sends warnings to authorities and affected people.

Rescue Coordination Agent

Suggests evacuation routes and resource allocation.

6. Example Decision Logic

IF rainfall is very high
AND river level is increasing
AND satellite indicates flooded region
THEN
 Flood Risk = HIGH
 Activate Alert Agent
 Recommend Evacuation

7. Scalability

PySpark distributes large satellite, sensor, and social-media datasets among multiple nodes.

Benefits:

- Parallel processing.
- High-speed analysis.
- Fault tolerance.
- Processing of large-scale data.
- Easy expansion by adding cluster nodes.

8. Processing Pipeline

Multi-source Data



Data Cleaning



RDD Partitioning



Feature Extraction



Data Fusion



AI Detection



Risk Classification



Agent Coordination



Early Warning



Emergency Response

9. Expected Outcome

The system provides faster disaster detection, reduces false alarms by combining multiple data sources, and supports automated early-warning and emergency-response decisions.

CODE :

```
from pyspark import SparkConf, SparkContext

conf = SparkConf().setAppName("SmartDisasterDetection").setMaster("local[*]")
sc = SparkContext(conf=conf)

# -----
# Multi-source sensor data
# -----

sensor_data = [
    ("S1", "Flood", 85),
    ("S2", "Flood", 90),
    ("S3", "Flood", 40),
    ("S4", "Earthquake", 75),
    ("S5", "Earthquake", 92),
    ("S6", "Earthquake", 30)
]

social_media = [
    ("Flood", "Heavy rainfall reported"),
    ("Flood", "River water level increasing"),
    ("Earthquake", "Strong shaking experienced"),
    ("Earthquake", "Buildings vibrating")
]

satellite_data = [
    ("Flood", 80),
    ("Flood", 90),
    ("Earthquake", 70),
    ("Earthquake", 85)]

sensor_rdd = sc.parallelize(sensor_data)
social_rdd = sc.parallelize(social_media)
satellite_rdd = sc.parallelize(satellite_data)

# -----
# Sensor analysis
# -----

sensor_average = (
    sensor_rdd
```

```

        .map(lambda x: (x[1], x[2]))
        .groupByKey()
        .mapValues(lambda values: sum(values) / len(values))
    )
# -----
# Satellite analysis
# -----
satellite_average = (
    satellite_rdd
    .groupByKey()
    .mapValues(lambda values: sum(values) / len(values))
)
# -----
# Social media count
# -----
social_count = (
    social_rdd
    .map(lambda x: (x[0], 1))
    .reduceByKey(lambda a, b: a + b)
)
# -----
# Data Fusion
# -----
fused_data = (
    sensor_average
    .join(satellite_average)
    .join(social_count)
    .map(lambda x: (
        x[0],
        x[1][0][0],
        x[1][0][1],
        x[1][1]
    ))
)
# -----
# Intelligent Disaster Agent

```

```
def disaster_agent(data):
    disaster, sensor, satellite, social = data
    risk_score = (
        sensor * 0.4 +
        satellite * 0.4 +
        social * 10 * 0.2
    )
    if risk_score >= 70:
        status = "HIGH RISK"
        action = "SEND EARLY WARNING"
    elif risk_score >= 50:
        status = "MEDIUM RISK"
        action = "MONITOR CLOSELY"
    else:
        status = "LOW RISK"
        action = "NO IMMEDIATE ACTION"
    return disaster, round(risk_score, 2), status, action
results = fused_data.map(disaster_agent).collect()
print("=== SMART DISASTER DETECTION SYSTEM ===")
for result in results:
    print("\nDisaster:", result[0])
    print("Risk Score:", result[1])
    print("Status:", result[2])
    print("Agent Action:", result[3])
sc.stop()
```

OUTPUT :

=== SMART DISASTER DETECTION SYSTEM ===

Disaster: Flood

Risk Score: 77.0

Status: HIGH RISK

Agent Action: SEND EARLY WARNING

Disaster: Earthquake

Risk Score: 72.8

Status: HIGH RISK

Agent Action: SEND EARLY WARNING

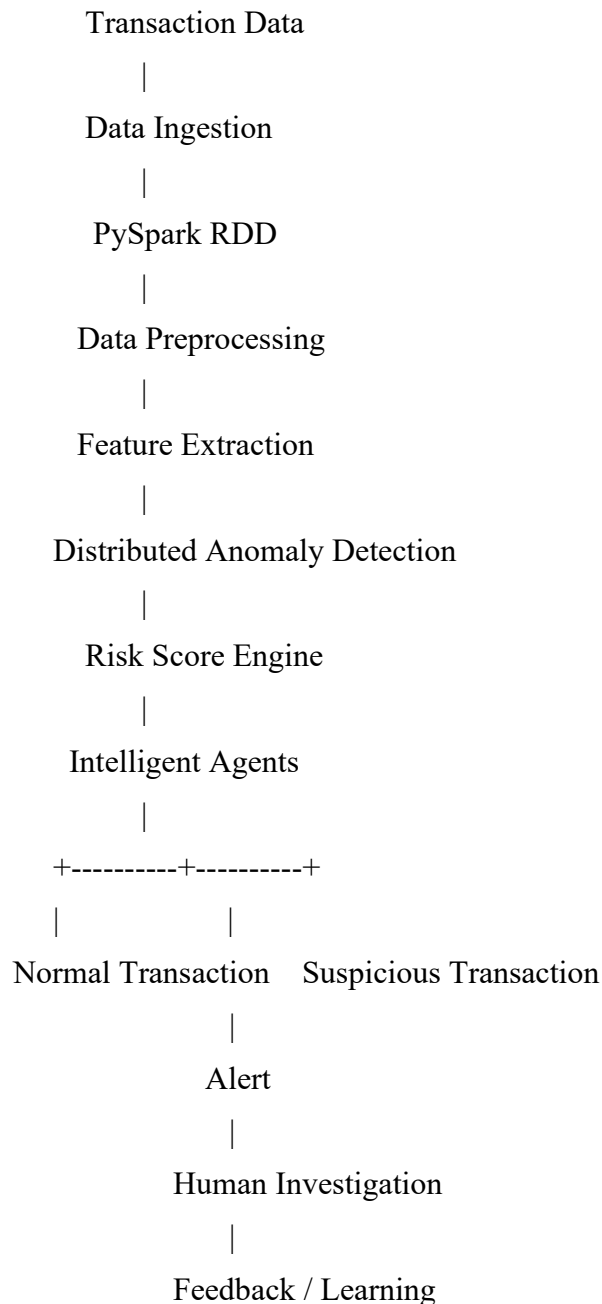
3. AI-Based Fraud Detection Framework :

1. Problem Understanding

Financial institutions process millions of transactions every day. Manual fraud detection is difficult because fraudulent transactions may be hidden among a huge number of legitimate transactions.

The proposed framework uses **PySpark RDDs, distributed anomaly detection, and intelligent agents.**

2. System Architecture



3. RDD Operations

```
from pyspark import SparkContext
sc = SparkContext("local", "FraudDetection")
transactions = sc.textFile("transactions.csv")
data = transactions.map(lambda x: x.split(","))
```

```
valid = data.filter(lambda x: float(x[2]) > 0)
amounts = valid.map(lambda x: (x[1], float(x[2])))
total = amounts.reduceByKey(lambda a, b: a + b)
result = total.collect()
```

RDD operations can process transaction records across multiple machines.

4. Anomaly Detection

Features can include:

- Transaction amount.
- Transaction frequency.
- Location.
- Time.
- Device information.
- Previous transaction behavior.
- Unusual spending patterns.

Anomaly score:

Low score → Normal

Medium score → Review

High score → Suspicious

The detection algorithm can operate on partitions in parallel.

5. Intelligent Agents

Transaction Monitoring Agent

Monitors incoming transactions.

Anomaly Detection Agent

Identifies unusual patterns.

Risk Assessment Agent

Calculates fraud probability.

Alert Agent

Generates alerts for suspicious transactions.

Investigation Agent

Collects supporting transaction information.

Learning Agent

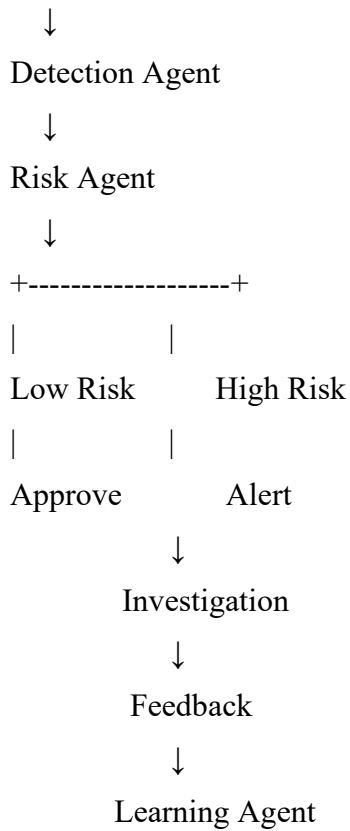
Uses confirmed fraud/legitimate feedback to improve future detection.

6. Agent Collaboration

Transaction

↓

Monitoring Agent

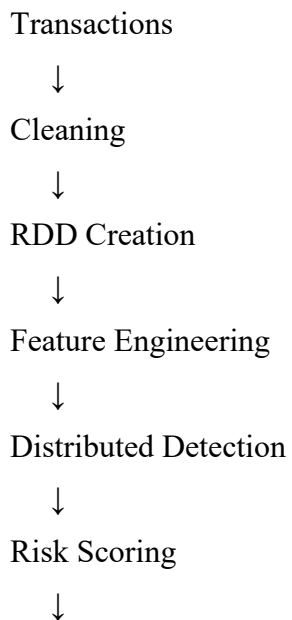


7. Scalability

PySpark enables:

- Parallel transaction processing.
- Distributed anomaly detection.
- Fault-tolerant execution.
- Processing of high-volume financial data.
- Horizontal scaling across cluster nodes.

8. Data Pipeline



Agent Decision



Alert / Approval



Feedback



Model Improvement

9. Expected Outcome

The framework detects suspicious transactions quickly and continuously improves through feedback. Distributed processing enables the system to operate efficiently even when transaction volume becomes very large.

CODE :

```
from pyspark import SparkConf, SparkContext
conf = SparkConf().setAppName("FraudDetection").setMaster("local[*]")
sc = SparkContext(conf=conf)
# -----
# Financial transaction data
# -----
transactions = [
    ("T001", "U1", 500, "Chennai"),
    ("T002", "U1", 700, "Chennai"),
    ("T003", "U2", 15000, "Mumbai"),
    ("T004", "U2", 20000, "Delhi"),
    ("T005", "U3", 400, "Chennai"),
    ("T006", "U3", 600, "Chennai"),
    ("T007", "U4", 25000, "London"),
    ("T008", "U4", 30000, "USA")
]
transaction_rdd = sc.parallelize(transactions)
# -----
# Calculate average transaction
# -----
user_average = (
    transaction_rdd
    .map(lambda x: (x[1], x[2]))
    .groupByKey()
    .mapValues(lambda values: sum(values) / len(values))
)
```

```

)
# -----
# Detect large transactions
# -----
large_transactions = (
    transaction_rdd
    .filter(lambda x: x[2] > 10000)
)
# -----
# Intelligent Fraud Agent
# -----
def fraud_agent(transaction, averages):
    transaction_id, user, amount, location = transaction
    average = averages.get(user, 0)
    if amount > 10000 and amount > average * 1.5:
        risk = "HIGH"
        action = "FRAUD ALERT"
    elif amount > 10000:
        risk = "MEDIUM"
        action = "REVIEW TRANSACTION"
    else:
        risk = "LOW"
        action = "APPROVE"
    return transaction_id, user, amount, location, risk, action
average_dict = user_average.collectAsMap()
results = (
    transaction_rdd
    .map(lambda x: fraud_agent(x, average_dict))
    .collect()
)
print("=== AI-BASED FRAUD DETECTION ===")
for result in results:
    print(
        result[0],
        "| User:", result[1],
        "| Amount:", result[2],

```

```
"| Location:", result[3],  
"| Risk:", result[4],  
"| Action:", result[5]  
)  
sc.stop()
```

OUTPUT :

=== AI-BASED FRAUD DETECTION ===

T001 | User: U1 | Amount: 500 | Location: Chennai | Risk: LOW | Action: APPROVE

T002 | User: U1 | Amount: 700 | Location: Chennai | Risk: LOW | Action: APPROVE

T003 | User: U2 | Amount: 15000 | Location: Mumbai | Risk: MEDIUM | Action: REVIEW TRANSACTION

T004 | User: U2 | Amount: 20000 | Location: Delhi | Risk: HIGH | Action: FRAUD ALERT

T005 | User: U3 | Amount: 400 | Location: Chennai | Risk: LOW | Action: APPROVE

T006 | User: U3 | Amount: 600 | Location: Chennai | Risk: LOW | Action: APPROVE

T007 | User: U4 | Amount: 25000 | Location: London | Risk: MEDIUM | Action: REVIEW TRANSACTION

T008 | User: U4 | Amount: 30000 | Location: USA | Risk: HIGH | Action: FRAUD ALERT

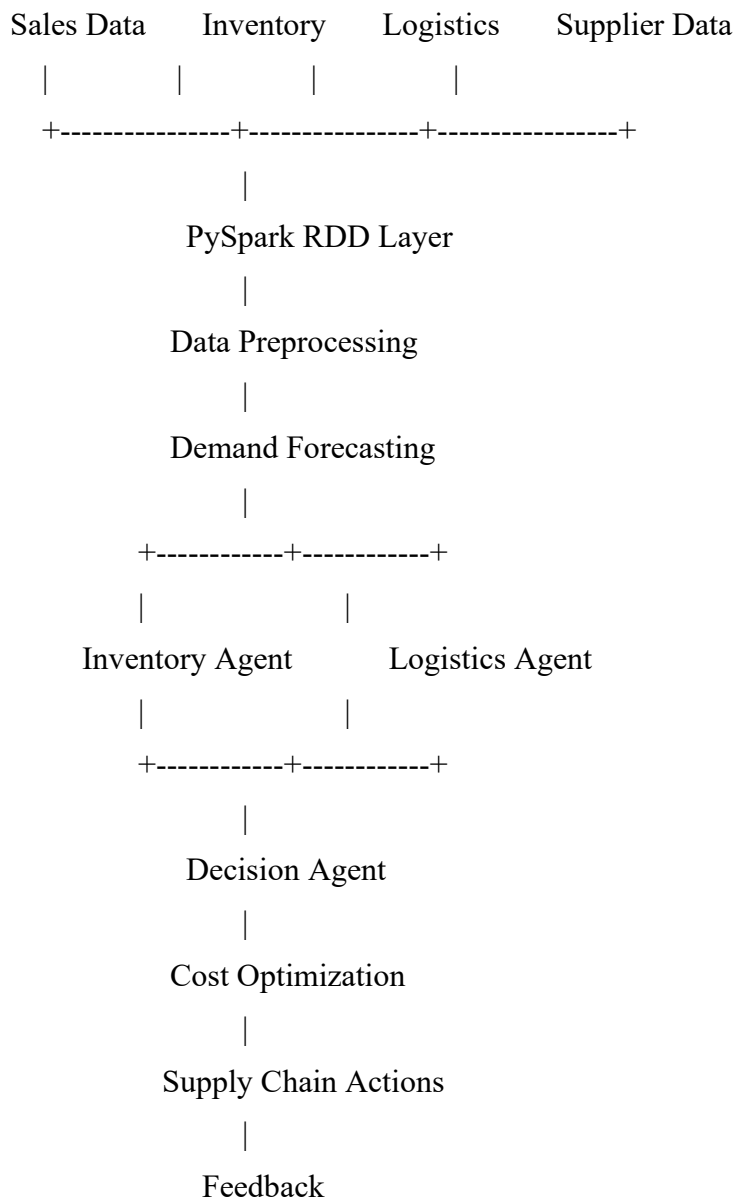
4. Autonomous Supply Chain Optimization System

1. Problem Understanding

Supply chains involve inventory management, transportation, demand forecasting, warehouses, suppliers, and customers. Poor coordination can cause overstocking, stockouts, transportation delays, and increased costs.

The proposed system uses PySpark and multiple intelligent agents to make decentralized supply-chain decisions.

2. System Architecture



3. RDD Transformations

```
from pyspark import SparkContext
sc = SparkContext("local", "SupplyChain")
sales = sc.textFile("sales.csv")
records = sales.map(lambda x: x.split(","))
products = records.map(lambda x: (x[0], int(x[2])))
```

```
demand = products.reduceByKey(lambda a, b: a + b)
```

```
result = demand.collect()
```

Important transformations include:

- `map()` for feature transformation.
- `filter()` for valid records.
- `flatMap()` for generating supply-demand records.
- `reduceByKey()` for demand aggregation.
- `join()` for combining inventory and demand data.

4. Intelligent Agents

Demand Forecasting Agent

Predicts future product demand.

Inventory Agent

Maintains appropriate stock levels.

Logistics Agent

Selects efficient transportation routes.

Supplier Agent

Monitors supplier availability and delivery times.

Cost Optimization Agent

Attempts to minimize total operational cost.

Decision Agent

Coordinates decisions among all agents.

5. Decentralized Decision Making

For example:

Demand increases

↓

Forecasting Agent detects increase

↓

Inventory Agent checks stock

↓

Supplier Agent checks availability

↓

Logistics Agent checks transportation

↓

Decision Agent selects optimal action

6. Cost Optimization

The system attempts to minimize:

Total Cost =
Inventory Cost
+ Transportation Cost
+ Ordering Cost
+ Shortage Cost

The agents continuously evaluate alternatives and select decisions that reduce overall cost.

7. Scalability

PySpark distributes sales, inventory, logistics, and supplier data across worker nodes.

Advantages:

- Parallel processing.
- Large-scale analytics.
- Fault tolerance.
- Faster demand aggregation.
- Easy cluster expansion.

8. Data Processing Pipeline

Raw Supply Chain Data



Cleaning



RDD Creation



Data Aggregation



Demand Forecasting



Inventory Analysis



Logistics Optimization



Multi-Agent Decision



Execution



Performance Feedback

9. Expected Outcome

The system reduces inventory shortages, transportation costs, delays, and unnecessary stock while improving supply-chain efficiency through distributed processing and autonomous intelligent agents.

CODE :

```
from pyspark import SparkConf, SparkContext

conf = SparkConf().setAppName("SupplyChainOptimization").setMaster("local[*]")
sc = SparkContext(conf=conf)
# -----
# Product inventory data
# -----
inventory = [
    ("Laptop", 20, 100),
    ("Mobile", 80, 60),
    ("Tablet", 15, 70),
    ("Monitor", 50, 40),
    ("Keyboard", 100, 20)
]
# Product, Current Stock, Daily Demand
inventory_rdd = sc.parallelize(inventory)
# -----
# Calculate stock requirement
# -----
stock_status = (
    inventory_rdd
    .map(lambda x: (
        x[0],
        x[1],
        x[2],
        x[1] - x[2]
    ))
)
# -----
# Intelligent Inventory Agent
# -----
def inventory_agent(item):
    product, stock, demand, balance = item
```

```

if stock < demand:
    decision = "REORDER"
    reorder_quantity = demand * 2 - stock
elif stock < demand * 1.5:
    decision = "ORDER SOON"
    reorder_quantity = demand - stock
else:
    decision = "STOCK SUFFICIENT"
    reorder_quantity = 0
return (
    product,
    stock,
    demand,
    decision,
    reorder_quantity
)
results = stock_status.map(inventory_agent).collect()
print("=== AUTONOMOUS SUPPLY CHAIN OPTIMIZATION ===")
for result in results:
    print("\nProduct:", result[0])
    print("Current Stock:", result[1])
    print("Daily Demand:", result[2])
    print("Agent Decision:", result[3])
    print("Order Quantity:", result[4])
sc.stop()

```

OUTPUT :

=== AUTONOMOUS SUPPLY CHAIN OPTIMIZATION ===

Product: Laptop

Current Stock: 20

Daily Demand: 100

Agent Decision: REORDER

Order Quantity: 180

Product: Mobile

Current Stock: 80

Daily Demand: 60

Agent Decision: ORDER SOON

Order Quantity: 0

Product: Tablet

Current Stock: 15

Daily Demand: 70

Agent Decision: REORDER

Order Quantity: 125

Product: Monitor

Current Stock: 50

Daily Demand: 40

Agent Decision: ORDER SOON

Order Quantity: 30

Product: Keyboard

Current Stock: 100

Daily Demand: 20

Agent Decision: STOCK SUFFICIENT

Order Quantity: 0

5. Personalized Learning Analytics Platform :

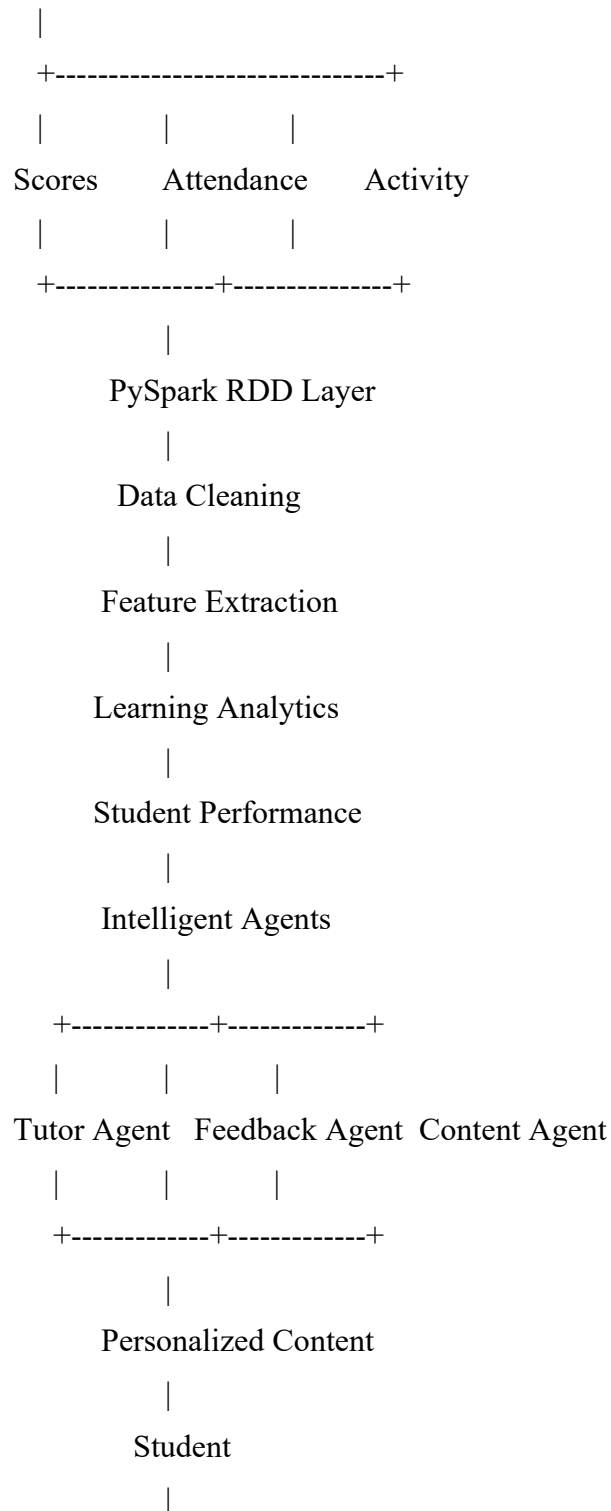
1. Problem Understanding

Educational platforms generate large amounts of student data such as attendance, quiz scores, assignment performance, video watching behavior, learning time, and interaction history.

The proposed system uses PySpark RDDs to analyze thousands of learners and intelligent agents to provide personalized learning.

2. System Architecture

Student Data



Feedback

|

Model Update

3. PySpark RDD Processing

```
from pyspark import SparkContext
sc = SparkContext("local", "LearningAnalytics")
data = sc.textFile("students.csv")
records = data.map(lambda x: x.split(","))
valid = records.filter(
    lambda x: float(x[2]) >= 0)
scores = valid.map(lambda x: (x[0], float(x[2])))
average_scores = scores.groupByKey().mapValues(lambda values: sum(values) / len(values))
result = average_scores.collect()
```

RDD operations such as map, filter, groupByKey, and reduceByKey can analyze student data in parallel.

4. Learning Analytics

The system evaluates:

- Quiz scores.
- Assignment marks.
- Attendance.
- Learning time.
- Number of attempts.
- Topics completed.
- Incorrect answers.
- Student interaction patterns.

A learner profile can be created from these features.

5. Intelligent Agents

Student Monitoring Agent

Tracks learning behavior.

Performance Analysis Agent

Identifies strengths and weaknesses.

Content Recommendation Agent

Recommends suitable learning materials.

Feedback Agent

Provides immediate feedback.

Difficulty Adaptation Agent

Changes question difficulty based on performance.

Instructor Agent

Provides useful analytics to teachers.

6. Adaptive Learning

Example:

Student attempts quiz



Performance Analysis



Score < 50%



Identify weak topic



Content Agent recommends basic material



Difficulty Agent reduces difficulty



Student practices



New performance data



Recommendation updated

If the student performs well, the system can recommend advanced learning material.

7. Scalability

PySpark enables the platform to analyze data from thousands of students simultaneously.

Benefits:

- Distributed processing.
- Parallel analytics.
- Fault tolerance.
- Fast processing of large datasets.
- Scalable student monitoring.

8. Real-Time Feedback

When a learner completes a quiz, the agents can immediately analyze the result and recommend the next activity.

For example:

Score = 40%

Weak Topic = Machine Learning

Recommendation = Beginner ML Tutorial

After improvement:

Score = 85%

Recommendation = Advanced ML Problems

9. Data Flow

Student Activity



Data Collection



RDD Processing



Feature Extraction



Performance Analysis



Student Profiling



Agent-Based Recommendation



Personalized Content



Student Feedback



Continuous Adaptation

10. Expected Outcome

The platform provides personalized learning paths, faster feedback, early identification of struggling students, and scalable analytics for thousands of learners. Intelligent agents continuously adapt content according to each student's performance and behavior.

CODE :

```
from pyspark import SparkConf, SparkContext
conf = SparkConf().setAppName("PersonalizedLearningAnalytics").setMaster("local[*]")
sc = SparkContext(conf=conf)
# -----
# Student learning data
# -----
students = [
    ("S1", "Python", 85, 90),
```



```

("S1", "AI", 65, 70),
("S2", "Python", 45, 50),
("S2", "AI", 40, 45),
("S3", "Python", 92, 95),
("S3", "AI", 88, 90),
("S4", "Python", 55, 60),
("S4", "AI", 50, 55)
]
# Student, Subject, Score, Engagement
student_rdd = sc.parallelize(students)
# -----
# Calculate average score
# -----
student_average = (
    student_rdd
    .map(lambda x: (x[0], x[2]))
    .groupByKey()
    .mapValues(lambda scores: sum(scores) / len(scores))
)
# -----
# Intelligent Learning Agent
# -----
def learning_agent(student_data):
    student, avg_score = student_data
    if avg_score >= 80:
        level = "ADVANCED"
        recommendation = "Provide advanced learning content"
    elif avg_score >= 60:
        level = "INTERMEDIATE"
        recommendation = "Provide practice exercises"
    else:
        level = "BEGINNER"
        recommendation = "Provide basic concepts and tutorials"
    return (
        student,
        round(avg_score, 2),

```

```
        level,  
        recommendation  
    )  
results = student_average.map(learning_agent).collect()  
print("=== PERSONALIZED LEARNING ANALYTICS ===")  
for result in results:  
    print("\nStudent:", result[0])  
    print("Average Score:", result[1])  
    print("Learning Level:", result[2])  
    print("Agent Recommendation:", result[3])  
sc.stop()
```

OUTPUT :

=== PERSONALIZED LEARNING ANALYTICS ===

Student: S1

Average Score: 75.0

Learning Level: INTERMEDIATE

Agent Recommendation: Provide practice exercises

Student: S2

Average Score: 42.5

Learning Level: BEGINNER

Agent Recommendation: Provide basic concepts and tutorials

Student: S3

Average Score: 90.0

Learning Level: ADVANCED

Agent Recommendation: Provide advanced learning content

Student: S4

Average Score: 52.5

Learning Level: BEGINNER

Agent Recommendation: Provide basic concepts and tutorials